Más Kilómetros

Functional & Backend Design Document

Author: Juan Alberto Gonzalez Bastida
Date: 2026
Version: 1.0

System Overview

Backend platform for sports race management (running and cycling), designed to facilitate event organization and user participation in a structured and secure manner.

The system allows users to register for events, manage their registrations, and perform secure payment operations. The platform supports full race administration for organizers, including race creation, category management, and participant capacity control.

Additionally, it manages the full registration lifecycle, from registration creation to payment confirmation, including automatic expiration of temporary reservations, payment retry workflows, and refund processing associated with cancellations.

The system implements explicit state management across entities, ensures transactional consistency in critical operations, and incorporates role-based access control while prioritizing data integrity and consistency.

Technical Stack

- Java
- Spring Boot
- Spring Security
- Spring Data JPA
- Hibernate
- MySQL
- Docker
- JWT Authentication
- REST API
- Spring AOP
- Docker Compose
- Logback

Roles and Responsibilities

The system defines different roles with specific responsibilities to ensure proper operational control and secure access to resources.

USER

- Register on the platform
- Authenticate (login)
- Browse available races
- Register for races
- View and manage their registrations
- Perform and retry registration payments
- Cancel registrations
- View payment history and payment status
- View race results

ORGANIZER

- Create and manage races
- Publish, close, and cancel races
- Configure participant capacity and availability
- Define race categories
- View registrations and participants by race
- Manage registered participants
- View basic participation metrics
- Upload race results manually or through file import

ADMIN

- Create, edit, and manage users
- Assign roles
- Full access to races, registrations, and payments
- Monitor payment and registration operations
- Manage system incidents
- Access global system information

Domain Model (Entities and Relationships)

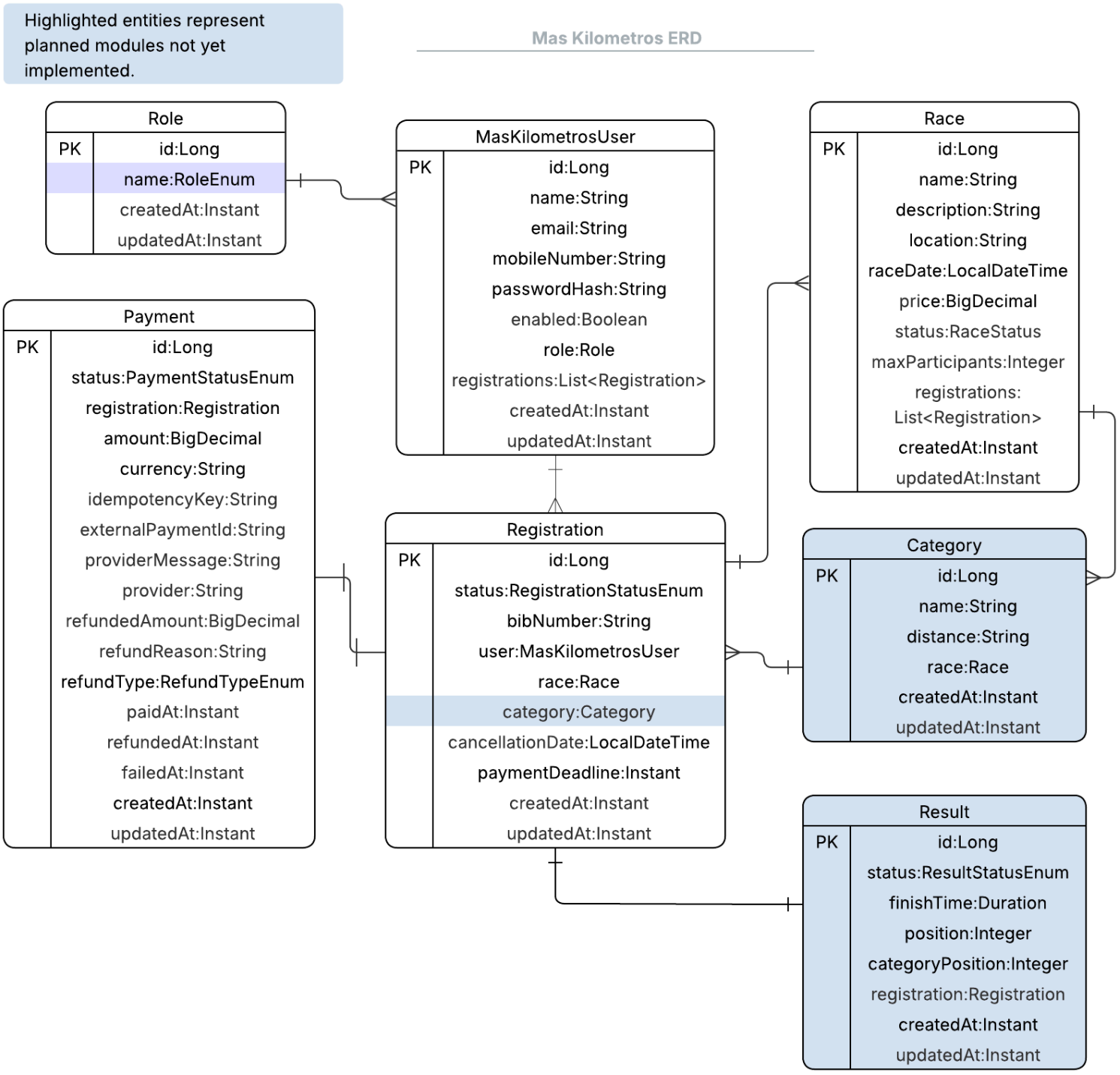
Overview of Entities

- User → represents a platform user
- Role → defines user permissions and access levels
- Race → represents a sports race event
- Category → represents a category within a race

- Registration → represents a user's registration to a race and manages the participation lifecycle
- Payment → represents the payment workflow associated with a registration, including retries and refunds
- Result → represents the final result of a participant in a race, associated with a valid registration

Entity Relationships (Diagram)

The system is centered around the Registration entity, which links users to races through categories. Payments and results are associated with each registration, ensuring proper tracking of participation and financial transactions.



Entity Details

MasKilometrosUser

Attributes:

- id → unique identifier
- name → user full name
- email → user email address
- mobileNumber → user mobile phone number
- passwordHash → securely hashed user password
- enabled → indicates whether the user account is enabled
- role (Role) → assigned user role
- registrations(List<Registration>) → registrations associated with the user
- createdAt → entity creation timestamp
- updatedAt → last entity update timestamp

Relationships:

- role → ManyToOne
- registrations → OneToMany

Role

Attributes:

- id → unique identifier
- name → role name and access level (USER, ORGANIZER, ADMIN)
- createdAt → entity creation timestamp
- updatedAt → last entity update timestamp

Race

Attributes:

- id → unique identifier
- name → race name
- description → race description
- location → race location
- raceDate → scheduled race date and time
- price → registration price for the race
- status → race lifecycle status (DRAFT, PUBLISHED, CLOSED, CANCELLED)
- maxParticipants → maximum allowed participants
- registrations → registrations associated with the race
- createdAt → entity creation timestamp
- updatedAt → last entity update timestamp

Relationships:

- Registration → OneToMany

Category

Attributes:

- id → unique identifier
- name → category name
- distance → race distance associated with the category (5K, 10K, etc.)
- race (Race) → assigned category race
- createdAt → entity creation timestamp
- updatedAt → last entity update timestamp

Relationships:

- race → ManyToOne

Registration

Attributes:

- id → unique identifier
- status → registration lifecycle status (PENDING_PAYMENT, PAID, CANCELLED)
- bibNumber → participant bib number assigned after successful payment
- user (MasKilometrosUser) → → registration owner
- race (Race) → assigned registration race
- category (Category) → assigned registration category
- cancellationDate → registration cancellation timestamp
- paymentDeadline → payment expiration deadline
- createdAt → entity creation timestamp
- updatedAt → last entity update timestamp

Relationships:

- user → ManyToOne
- race → ManyToOne
- category → ManyToOne
- payment → OneToOne
- result → OneToOne

Payment

Attributes:

- id → unique identifier
- status → payment lifecycle status (PENDING, COMPLETED, FAILED, REFUNDED)
- registration (Registration) → assigned payment registration
- amount → payment amount
- currency → payment currency
- idempotencyKey → unique key to prevent duplicate charges
- externalPaymentId → external payment provider transaction identifier
- providerMessage → payment provider response message
- provider → payment provider used to process the transaction

- refundedAmount → refunded amount processed by the system
- refundReason → refund justification
- refundType → refund policy applied (FULL, PARTIAL, NONE)
- paidAt → payment confirmation timestamp
- refundedAt → refund confirmation timestamp
- failedAt → failed payment timestamp
- createdAt → entity creation timestamp
- updatedAt → last entity update timestamp

Relationships:
- registration → OneToOne

Result

Attributes:
- id → unique identifier
- status → participant race result status (FINISHED, DNF, DSQ)
- finishTime → participant total finish time
- position → overall race position
- categoryPosition → position within category
- registration (Registration) → assigned result registration
- createdAt → entity creation timestamp
- updatedAt → last entity update timestamp

Relationships:
- registration → OneToOne

Core Workflows

User Registration

Happy Path
1. User submits registration data (name, email, password, etc.)
2. System validates required fields and input format (e.g., valid email)
3. System checks if a user with the same email already exists
4. System validates password strength
5. System hashes the password securely
6. System creates a new User entity
7. System assigns default role (USER)
8. System persists the user in the database
9. System returns a success response

Alternative Flows
- Email already exists
    1. System detects duplicate email
    2. System returns error response (409 Conflict)

- Invalid input data
    1. System detects missing or invalid fields
    2. System returns validation error (400 Bad Request)

- Weak password
    1. System rejects password based on security rules
    2. System returns validation error

- System error during persistence
    1. Database operation fails
    2. System returns internal error (500)

Authentication

Happy Path

1. User submits credentials (email and password)
2. System validates input data
3. System retrieves user by email
4. System verifies that the user exists and is active
5. System compares the provided password with the stored hashed password
6. System generates a JWT token containing user information (e.g., userId, role)
7. System signs the token securely
8. System returns a successful response with the JWT token

Alternative Flows

- User not found
    1. System cannot find user by email
    2. System returns authentication error (401 Unauthorized)

- Invalid password
    1. Password comparison fails
    2. System returns authentication error (401 Unauthorized)

- Inactive or blocked user
    1. - User status is not valid for login
    2. System returns forbidden error (403)

- Invalid input data
    1. Missing or malformed credentials
    2. System returns validation error (400 Bad Request)

Registration and Payment

Happy Path

1. Authenticated user selects a race and a category

2. System validates that the race exists and is in PUBLISHED state
3. System validates that the selected category belongs to the race
4. System checks that the user is not already registered for the same race
5. System creates a Registration entity with status PENDING_PAYMENT
6. User initiates the payment process
7. System creates a Payment entity with status PENDING and generates an idempotencyKey
8. System processes the payment request through the configured payment provider
9. Payment provider processes the request and returns SUCCESS
10. System validates provider response (including idempotency)
11. System updates Payment → COMPLETED and sets paidAt timestamp
12. System updates Registration → PAID
13. System assigns bibNumber
14. System triggers a confirmation notification (email/event)
15. System returns a successful response to the user

Alternative Flows

- User already registered
    1. System detects an existing Registration for the same user and race
    2. System rejects the request
    3. System returns error response (409 Conflict)

- Race not available
    1. Race is not in PUBLISHED state or is closed
    2. System rejects the registration
    3. System returns validation error (400 Bad Request)

- Invalid category
    1. Selected category does not belong to the race
    2. System rejects the request
    3. System returns validation error (400 Bad Request)

- Payment failure
    1. Payment provider returns FAILED
    2. System updates Payment → FAILED
    3. Registration remains in PENDING_PAYMENT
    4. User can retry payment

- Duplicate payment attempt (idempotency)
    1. Same idempotencyKey is received
    2. System detects existing Payment
    3. System returns previous result without creating a new charge

- Payment provider timeout / delayed response (Future asynchronous provider behavior)
    1. Provider does not respond immediately
    2. Payment remains in PENDING state

3. System may allow payment retry attempts

- System failure after payment success
    1. Payment is successful but system fails before updating Registration
    2. System transaction management ensures consistency between Payment and Registration states

Cancellation

This flow describes how a user cancels a race registration and how the system handles potential refunds.

Happy Path

1. Authenticated user requests cancellation of a registration
2. System retrieves the Registration
3. System validates that the registration exists and belongs to the user
4. System validates that the registration is eligible for cancellation:
    - Status is PAID or PENDING_PAYMENT
    - Race has not started
    - Cancellation policy allows refund processing according to race cancellation rules

5. If Registration is PENDING_PAYMENT:
    - System updates Registration → CANCELLED
    - No payment action required

6. If Registration is PAID:
    - System processes refund according to cancellation policy
    - System updates Payment → REFUNDED
    - System stores refund information and timestamps
    - System updates Registration → CANCELLED

7. System triggers cancellation confirmation notification
8. System returns success response

Alternative Flows

- Registration not found
    1. Registration does not exist
    2. System returns error (404 Not Found)

- Unauthorized access
    1. Registration does not belong to user
    2. System returns error (403 Forbidden)

- Cancellation not allowed
    1. Race already started OR cancellation window expired

2. System rejects request
3. System returns validation error (400 Bad Request)

- Refund processing failure
    1. System fails during refund processing
    2. Payment state remains unchanged
    3. Registration cancellation is rejected
    4. System returns error response

- System failure during refund
    1. Refund processed but system fails before updating DB

Results

This flow describes how race results are processed and made available to users.

Happy Path (Result Upload)

1. Organizer uploads results file (or inputs data manually)
2. System validates file format and data integrity
3. System processes each record:
    - Identifies Registration (via bibNumber or ID)
    - Validates Registration is in PAID status
4. System creates Result entity:
    - finishTime
    - position
    - categoryPosition
    - status (FINISHED)
5. System saves results for all valid registrations
6. System marks results as available for the race
7. System returns success response

Happy Path (Result Retrieval)

1. User requests results for a race or personal results
2. System retrieves results associated with user's registrations
3. System returns results (time, position, categoryPosition)

Alternative Flows

- Invalid file format
    1. File structure is incorrect
    2. System rejects upload
    3. System returns validation error (400 Bad Request)

- Registration not found
    1. Result entry does not match any Registration
    2. System skips or logs error

- Registration not valid for results
    1. Registration is not PAID
    2. System ignores result entry

- Duplicate result
    1. Result already exists for registration
    2. System rejects or updates based on policy

Business Rules

- A user cannot register more than once for the same race
- Only registrations in PAID status are considered confirmed participants
- A race must be in PUBLISHED state to allow new registrations
- A user must select a valid category that belongs to the selected race
- A registration cannot be created if the race has already started or is closed
- Payments must be idempotent to prevent duplicate charges
- A registration in PENDING_PAYMENT status can be cancelled without refund
- A registration in PAID status can only be cancelled within the allowed cancellation window (according to the configured refund policy)
- A paid registration cancellation triggers a refund process
- A registration cannot be cancelled after the race has started
- Results can only be assigned to registrations with PAID status
- Each registration can have at most one result
- A result must be associated with a valid registration
- A bibNumber must be unique per race
- Only ORGANIZER or ADMIN roles can create or manage races
- Only authenticated users can register for races and perform payments
- Users can only access and manage their own registrations
- Refund operations must be linked to the original payment transaction

Future Enhancements

- External payment provider integration
- Real external payment provider integration (Stripe/MercadoPago)
- Webhook processing
- Notification service
- Race results import via CSV
- Advanced reporting and analytics
- Financial reconciliation workflows